

Encapsulation and Data Abstraction

Encapsulation

Encapsulation is the wrapping of data (variables) and functions (methods) together into a single unit called a class. The data inside the class is hidden from the outside world and can only be accessed through the functions defined within the class.

This prevention of direct data access from outside is called **data hiding** or **information hiding**.

Why Encapsulation Matters

Without encapsulation, any part of a program could modify any piece of data freely — leading to bugs that are hard to trace. Encapsulation ensures that:

- Data can only be changed in controlled, predictable ways
- The internal implementation can change without affecting the rest of the program
- The class acts as a protective wrapper around sensitive data

Example

```
class Employee {
private:
    double salary;           // hidden – cannot be accessed directly outside

public:
    void setSalary(double s) {
        if (s > 0) salary = s;    // controlled access
    }
    double getSalary() {
        return salary;
    }
};

int main() {
    Employee e;
    e.setSalary(75000);
    cout << e.getSalary();    // accessing through a controlled method
    // e.salary = -5000;      // ERROR – this would be blocked
}
```

The salary is private, so it cannot be set to an invalid negative value directly. The setSalary function acts as a gatekeeper.

Data Abstraction

Abstraction means representing only the essential features of an object while hiding the background implementation details.

A class uses abstraction by exposing a clean public interface (what the object can do) while keeping the internal mechanics hidden (how it does it).

Real-World Analogy

When you drive a car, you interact with the steering wheel, pedals, and gear shift. You do not need to know how the engine converts fuel into motion — that complexity is abstracted away. The car's interface (controls) is what you see; the implementation (engine mechanics) is hidden.

Abstraction in C++

```
class Stack {
private:
    int data[100];    // internal implementation hidden
    int top;

public:
    void push(int x);    // interface – what you can do
    int pop();
    bool isEmpty();
};
```

The user of this Stack class only needs to know about push, pop, and isEmpty — not how the array and top pointer work internally.

Encapsulation vs Abstraction

	Encapsulation	Abstraction
Focus	Hiding data	Hiding complexity
Achieved by	Access specifiers (private, public)	Abstract classes and interfaces
Purpose	Protects data from unauthorised access	Simplifies interaction with objects
What is hidden	Internal data	Internal implementation